PADEE.AI

Frontend + Backend Implementation Guide

Complete build guide — feature by feature — VS Code + Claude

Each section covers ONE feature end-to-end: backend route first, then the React screen that calls it.

Frontend	Backend	Auth	Duration
React 18 + Vite 4 + TypeScript + Tailwind + Framer Motion	Hono + LiteLLM + Supabase	Supabase Auth + JWT	8 weeks, solo founder

The one rule for every feature in this guide

1. Write the backend route. 2. Test with curl. 3. Add function to api.ts. 4. Wire the screen. Do not skip steps. Do not build frontend and backend at the same time.

Part 1 -- Project Setup (Day 1-2)

Before building any feature, the project needs: folder structure, Supabase client in both frontend and backend, and a working auth context in React.

1.1 -- Frontend: install dependencies

```
npm create vite@latest padee-ai -- --template react-ts
cd padee-ai

# Core deps
npm install react-router-dom@6 framer-motion
npm install @supabase/supabase-js
npm install tailwindcss @tailwindcss/vite

# Dev deps
npm install -D @types/react @types/react-dom typescript
```

1.2 -- Frontend: src/lib/supabaseClient.ts

Create this file once. Every screen imports from here. The anon key is safe to expose in the frontend.

```
import { createClient } from '@supabase/supabase-js'

const url = import.meta.env.VITE_SUPABASE_URL
const key = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = createClient(url, key)
```

1.3 -- Frontend: src/context/AuthContext.tsx

Wrap the app in this. Every screen can call useAuth() to get the current user and their JWT token.

```
import { createContext, useContext, useEffect, useState } from 'react'
import { supabase } from '../lib/supabaseClient'
import type { Session, User } from '@supabase/supabase-js'

interface AuthCtx {
  user: User | null
  session: Session | null
  token: string | null
  loading: boolean
  signOut: () => Promise<void>
}

const AuthContext = createContext<AuthCtx>({} as AuthCtx)

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [session, setSession] = useState<Session | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session)
      setLoading(false)
    })
    const { data: { subscription } } = supabase.auth.onAuthStateChange((_e, s) => {
```

```
      setSession(s)
    })
    return () => subscription.unsubscribe()
  }, [])

  return (
    <AuthContext.Provider value={{
      user: session?.user ?? null,
      session,
      token: session?.access_token ?? null,
      loading,
      signOut: () => supabase.auth.signOut()
    }}>
      {children}
    </AuthContext.Provider>
  )
}

export const useAuth = () => useContext(AuthContext)
```

1.4 -- Frontend: src/App.tsx with protected routes

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'
import { AuthProvider, useAuth } from './context/AuthContext'
import LoginScreen from './screens/Login'
import OnboardingScreen from './screens/Onboarding'
import HomeScreen from './screens/Home'
import AskAIScreen from './screens/AskAI'
import WorksheetScreen from './screens/Worksheet'

function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const { user, loading } = useAuth()
  if (loading) return <div className='loading-spinner' />
  if (!user) return <Navigate to='/login' replace />
  return <>{children}</>
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path='/login' element={<LoginScreen />} />
          <Route path='/onboarding' element={<OnboardingScreen />} />
          <Route path='/' element={<ProtectedRoute><HomeScreen /></ProtectedRoute>}
/>
          <Route path='/ask' element={<ProtectedRoute><AskAIScreen
/></ProtectedRoute>} />
          <Route path='/worksheet' element={<ProtectedRoute><WorksheetScreen
/></ProtectedRoute>} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  )
}
```

1.5 -- Frontend: src/services/api.ts (complete file)

This is the only file that talks to the backend. All screens import from here. Never call fetch() directly in a screen file.

```
const BASE = '/api'

// -- Auth helper --
function authHeader(token: string) {
```

```
  return { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' }
}

// -- Doubt: streaming SSE --
export async function askDoubt(
  token: string,
  messages: { role: string; content: string }[],
  subject: string,
  className: number,
  onChunk: (text: string) => void,
  onDone: () => void
): Promise<void> {
  const resp = await fetch(`${BASE}/ai/doubt`, {
    method: 'POST',
    headers: authHeader(token),
    body: JSON.stringify({ messages, subject, className }),
  })
  if (resp.status === 429) throw new Error('daily_limit_reached')
  if (!resp.ok) throw new Error(`HTTP ${resp.status}`)
  const reader = resp.body!.getReader()
  const decoder = new TextDecoder()
  while (true) {
    const { done, value } = await reader.read()
    if (done) break
    for (const line of decoder.decode(value).split('\n')) {
      if (line === 'data: [DONE]') { onDone(); continue }
      if (line.startsWith('data: ')) {
        try { onChunk(JSON.parse(line.slice(6)).text || '') } catch {}
      }
    }
  }
}

// -- Daily usage count --
export async function getDailyUsage(token: string): Promise<{ count: number; limit:
number }> {
  const r = await fetch(`${BASE}/ai/usage`, { headers: authHeader(token) })
  return r.json()
}

// -- Quality signals --
export async function submitFeedback(
  token: string, sessionId: string, helpful: boolean, reason?: string
) {
  await fetch(`${BASE}/ai/feedback`, { method: 'POST',
    headers: authHeader(token),
    body: JSON.stringify({ sessionId, helpful, reason }) })
}

export async function reportIncorrect(token: string, data: {
  sessionId?: string; questionText: string; aiResponse: string;
  subject: string; classLevel: number; reportText: string
}) {
  await fetch(`${BASE}/ai/flag`, { method: 'POST',
    headers: authHeader(token), body: JSON.stringify(data) })
}

// -- Visual explain --
export async function generateVisual(
  token: string, concept: string, subject: string, className: number
): Promise<string> {
  const r = await fetch(`${BASE}/ai/visual`, { method: 'POST',
    headers: authHeader(token),
    body: JSON.stringify({ concept, subject, className }) })
  return (await r.json()).html || ''
}

// -- Practice --
export async function generatePractice(token: string, data: {
```

```
  subject: string; className: number; chapter: string;
  count: number; difficulty: string
}) {
  const r = await fetch(`${BASE}/ai/practice`, { method: 'POST',
    headers: authHeader(token), body: JSON.stringify(data) })
  return r.json()
}

// -- Evaluate descriptive answer --
export async function evaluateAnswer(token: string, data: {
  question: string; studentAnswer: string; marks: number; subject: string
}) {
  const r = await fetch(`${BASE}/ai/evaluate`, { method: 'POST',
    headers: authHeader(token), body: JSON.stringify(data) })
  return r.json()
}

// -- Worksheet --
export async function generateWorksheet(token: string, data: {
  subject: string; className: number; chapter: string;
  questionTypes: string[]; difficulty: string; totalQuestions: number
}) {
  const r = await fetch(`${BASE}/ai/worksheet`, { method: 'POST',
    headers: authHeader(token), body: JSON.stringify(data) })
  return r.json()
}

// -- Paper Mimic --
export async function uploadMimicPDF(
  token: string, file: File, subject: string,
  className: number, questionCount: number
) {
  const form = new FormData()
  form.append('pdf', file)
  form.append('subject', subject)
  form.append('className', className.toString())
  form.append('questionCount', questionCount.toString())
  const r = await fetch(`${BASE}/ai/mimic`, {
    method: 'POST', headers: { Authorization: `Bearer ${token}` }, body: form })
  return r.json()
}

// -- Student profile --
export async function getProfile(token: string) {
  const r = await fetch(`${BASE}/user/profile`, { headers: authHeader(token) })
  return r.json()
}

export async function saveOnboarding(token: string, data: {
  className: number; subjects: string[]; track: string
}) {
  const r = await fetch(`${BASE}/user/onboarding`, { method: 'POST',
    headers: authHeader(token), body: JSON.stringify(data) })
  return r.json()
}
```

Part 2 -- Auth and Login (Day 2-3)

Build login first. Nothing else works without a valid JWT token.

2.1 -- Backend: POST /api/auth/login

Supabase Auth handles all authentication. The backend does not need a login route -- the frontend calls Supabase Auth directly. The backend only validates the JWT on protected routes.

There is no login route to build on the backend

Supabase Auth is called directly from the frontend using the @supabase/supabase-js client. The backend only needs getUserFromToken() which is already in server/lib/supabase.ts. Skip to the frontend login screen.

2.2 -- Frontend: src/screens/Login.tsx

```
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { supabase } from '../lib/supabaseClient'

export default function LoginScreen() {
  const nav = useNavigate()
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [schoolCode, setSchoolCode] = useState('')
  const [error, setError] = useState('')
  const [loading, setLoading] = useState(false)
  const [mode, setMode] = useState<'student' | 'teacher'>('student')

  async function handleLogin(e: React.FormEvent) {
    e.preventDefault()
    setError('')
    setLoading(true)
    try {
      const { error } = await supabase.auth.signInWithPassword({ email, password })
      if (error) throw error
      // Check if profile exists and has class selected
      const { data: profile } = await supabase
        .from('profiles')
        .select('class_level')
        .single()
      nav(profile?.class_level ? '/' : '/onboarding')
    } catch (err: any) {
      setError(err.message || 'Login failed')
    } finally {
      setLoading(false)
    }
  }

  async function handleSignup(e: React.FormEvent) {
    e.preventDefault()
    setLoading(true)
    const { error } = await supabase.auth.signUp({
      email, password,
      options: { data: { role: mode, school_code: schoolCode } }
    })
    if (error) setError(error.message)
```

```
    else nav('/onboarding')
    setLoading(false)
  }

  return (
    <div className='min-h-screen flex items-center justify-center bg-gray-50'>
      <div className='bg-white p-8 rounded-2xl shadow-sm border w-full max-w-sm'>
        <h1 className='text-2xl font-semibold text-teal-600 mb-1'>Padee.ai</h1>
        <p className='text-sm text-gray-500 mb-6'>Your AI study partner</p>

        <div className='flex gap-2 mb-6'>
          {(['student','teacher'] as const).map(m => (
            <button key={m} onClick={() => setMode(m)}
              className={`flex-1 py-2 rounded-lg text-sm font-medium transition-
colors
                ${mode === m ? 'bg-teal-50 text-teal-700 border border-teal-200'
                  : 'text-gray-500 hover:bg-gray-50'}`}>
              {m.charAt(0).toUpperCase() + m.slice(1)}
            </button>
          ))}
        </div>

        <form onSubmit={handleLogin} className='flex flex-col gap-3'>
          <input type='email' placeholder='Email' value={email}
            onChange={e => setEmail(e.target.value)}
            className='border rounded-lg px-3 py-2 text-sm focus:outline-none
focus:ring-2 focus:ring-teal-300' />
          <input type='password' placeholder='Password' value={password}
            onChange={e => setPassword(e.target.value)}
            className='border rounded-lg px-3 py-2 text-sm focus:outline-none
focus:ring-2 focus:ring-teal-300' />
          {error && <p className='text-red-500 text-xs'>{error}</p>}
          <button type='submit' disabled={loading}
            className='bg-teal-600 text-white rounded-lg py-2 text-sm font-medium
              hover:bg-teal-700 disabled:opacity-50 transition-colors'>
            {loading ? 'Signing in...' : 'Sign in'}
          </button>
          <button type='button' onClick={handleSignup} disabled={loading}
            className='text-teal-600 text-sm hover:underline'>
            Create account
          </button>
        </form>
      </div>
    </div>
  )
}
```

Part 3 -- Onboarding (Day 3-4)

The onboarding screen sets the student's class, subjects, and track. This data is used by the AI to personalise every response. Must be completed before the home screen loads.

3.1 -- Backend: POST /api/user/onboarding

```
// In server/routes/user.ts -- add this route
user.post('/onboarding', async (c) => {
  const u = await getUserFromToken(c.req.header('Authorization') ?? null)
  if (!u) return c.json({ error: 'Unauthorized' }, 401)

  const { className, subjects, track } = await c.req.json()

  // Update profile with class and track
  await supabase.from('profiles').update({
    class_level: className,
    active_track: track,
  }).eq('id', u.id)

  // Replace all subject selections
  await supabase.from('student_subjects').delete().eq('student_id', u.id)
  if (subjects.length > 0) {
    await supabase.from('student_subjects').insert(
      subjects.map((s: string) => ({ student_id: u.id, subject_code: s }))
    )
  }

  return c.json({ ok: true })
})
```

Test this before building the frontend

curl -X POST http://localhost:3001/api/user/onboarding -H 'Authorization: Bearer YOUR_JWT' -H 'Content-Type: application/json' -d '{"className":10,"subjects":["Physics","Chemistry"],"track":"school"}' -- should return {ok:true}

3.2 -- Frontend: src/screens/Onboarding.tsx (3-step flow)

```
import { useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'
import { saveOnboarding } from '../services/api'

const SUBJECTS_BY_CLASS: Record<number, string[]> = {
  8:  ['Physics','Chemistry','Biology','Maths','Social Science','English','Computer
Science'],
  9:  ['Physics','Chemistry','Biology','Maths','Social Science','English','Computer
Science'],
  10: ['Physics','Chemistry','Biology','Maths','Social Science','English','Computer
Science'],
  11: ['Physics','Chemistry','Biology','Maths','English','Computer
Science','Economics','Business Studies'],
  12: ['Physics','Chemistry','Biology','Maths','English','Computer
Science','Economics','Business Studies','Accounts'],
}

const TRACKS = [
```

```
  { id:'school',   label:'School learning',   sub:'CBSE boards focus' },
  { id:'foundation', label:'Foundation prep', sub:'JEE / NEET early prep' },
  { id:'ca',        label:'CA Foundation',    sub:'Chartered Accountancy' },
]

export default function OnboardingScreen() {
  const { token } = useAuth()
  const nav = useNavigate()
  const [step, setStep] = useState(1)
  const [className, setClassName] = useState<number | null>(null)
  const [selectedSubjects, setSelectedSubjects] = useState<string[]>([])
  const [track, setTrack] = useState('school')
  const [saving, setSaving] = useState(false)

  function toggleSubject(s: string) {
    setSelectedSubjects(prev =>
      prev.includes(s) ? prev.filter(x => x !== s) : [...prev, s]
    )
  }

  async function finish() {
    if (!token || !className) return
    setSaving(true)
    await saveOnboarding(token, { className, subjects: selectedSubjects, track })
    nav('/')
  }

  return (
    <div className='min-h-screen bg-gray-50 flex items-center justify-center p-4'>
      <div className='bg-white rounded-2xl p-8 w-full max-w-lg shadow-sm border'>

        {/* Step indicator */}
        <div className='flex gap-2 mb-8'>
          {[1,2,3].map(i => (
            <div key={i} className={`h-1.5 flex-1 rounded-full transition-colors
              ${step >= i ? 'bg-teal-500' : 'bg-gray-200'}`} />
          ))}
        </div>

        {/* Step 1: Class selection */}
        {step === 1 && (
          <div>
            <h2 className='text-xl font-semibold mb-2'>What class are you in?</h2>
            <p className='text-sm text-gray-500 mb-6'>This sets your CBSE
syllabus</p>
            <div className='flex gap-3 flex-wrap'>
              {[8,9,10,11,12].map(c => (
                <button key={c} onClick={() => setClassName(c)}
                  className={`px-5 py-2.5 rounded-xl text-sm font-medium border
transition-all
                    ${className === c
                      ? 'bg-teal-50 border-teal-400 text-teal-700'
                      : 'border-gray-200 hover:border-gray-300'}`}>
                  Class {c}
                </button>
              ))}
            </div>
            <button onClick={() => setStep(2)} disabled={!className}
              className='mt-8 w-full bg-teal-600 text-white py-2.5 rounded-xl
                font-medium disabled:opacity-40 hover:bg-teal-700'>
              Continue
            </button>
          </div>
        )}

        {/* Step 2: Subjects */}
        {step === 2 && className && (
          <div>
            <h2 className='text-xl font-semibold mb-2'>Pick your subjects</h2>
```

```
            <p className='text-sm text-gray-500 mb-6'>Select all you study</p>
            <div className='flex flex-wrap gap-2'>
              {SUBJECTS_BY_CLASS[className].map(s => (
                <button key={s} onClick={() => toggleSubject(s)}
                  className={`px-4 py-2 rounded-xl text-sm border transition-all
                    ${selectedSubjects.includes(s)
                      ? 'bg-teal-50 border-teal-400 text-teal-700'
                      : 'border-gray-200 hover:border-gray-300'}`}>
                  {s}
                </button>
              ))}
            </div>
            <button onClick={() => setStep(3)} disabled={selectedSubjects.length ===
0}
              className='mt-8 w-full bg-teal-600 text-white py-2.5 rounded-xl
                font-medium disabled:opacity-40 hover:bg-teal-700'>
              Continue
            </button>
          </div>
        )}

        {/* Step 3: Track */}
        {step === 3 && (
          <div>
            <h2 className='text-xl font-semibold mb-2'>What is your main goal?</h2>
            <p className='text-sm text-gray-500 mb-6'>You can change this
anytime</p>
            <div className='flex flex-col gap-3'>
              {TRACKS.map(t => (
                <button key={t.id} onClick={() => setTrack(t.id)}
                  className={`text-left p-4 rounded-xl border transition-all
                    ${track === t.id
                      ? 'bg-teal-50 border-teal-400'
                      : 'border-gray-200 hover:border-gray-300'}`}>
                  <p className='font-medium text-sm'>{t.label}</p>
                  <p className='text-xs text-gray-500 mt-0.5'>{t.sub}</p>
                </button>
              ))}
            </div>
            <button onClick={finish} disabled={saving}
              className='mt-8 w-full bg-teal-600 text-white py-2.5 rounded-xl
                font-medium disabled:opacity-40 hover:bg-teal-700'>
              {saving ? 'Setting up...' : 'Start learning'}
            </button>
          </div>
        )}
      </div>
    </div>
  )
}
```

Part 4 -- Ask AI Screen (Week 2 -- the most important feature)

This is the product. Get this working first before any other screen. The streaming hook is the hardest part -- once it works, everything else is straightforward.

4.1 -- Custom hook: src/hooks/useDoubtStream.ts

Extract the streaming logic into a reusable hook. The screen just calls this hook and renders the messages array.

```
import { useState, useCallback } from 'react'
import { askDoubt } from '../services/api'

export interface Message {
  id: string
  role: 'user' | 'assistant'
  content: string
  streaming?: boolean
  sessionId?: string
}

export function useDoubtStream(token: string | null, subject: string, className:
number) {
  const [messages, setMessages] = useState<Message[]>([])
  const [isStreaming, setIsStreaming] = useState(false)
  const [limitReached, setLimitReached] = useState(false)
  const [error, setError] = useState<string | null>(null)

  const sendDoubt = useCallback(async (content: string) => {
    if (!token || isStreaming) return
    setError(null)

    // Add user message
    const userMsg: Message = { id: Date.now().toString(), role: 'user', content }
    setMessages(prev => [...prev, userMsg])

    // Add empty assistant message for streaming
    const assistantId = (Date.now() + 1).toString()
    setMessages(prev => [...prev, {
      id: assistantId, role: 'assistant', content: '', streaming: true
    }])
    setIsStreaming(true)

    try {
      const history = [...messages, userMsg].map(m => ({
        role: m.role, content: m.content
      }))

      await askDoubt(
        token, history, subject, className,
        // onChunk: append to the streaming message
        (chunk) => setMessages(prev => prev.map(m =>
          m.id === assistantId
            ? { ...m, content: m.content + chunk }
            : m
        )),
        // onDone: mark as complete
        () => setMessages(prev => prev.map(m =>
          m.id === assistantId ? { ...m, streaming: false } : m
```

```
        ))
      )
    } catch (err: any) {
      if (err.message === 'daily_limit_reached') {
        setLimitReached(true)
        setMessages(prev => prev.filter(m => m.id !== assistantId))
      } else {
        setError('Something went wrong. Try again.')
        setMessages(prev => prev.filter(m => m.id !== assistantId))
      }
    } finally {
      setIsStreaming(false)
    }
  }, [token, messages, subject, className, isStreaming])

  return { messages, isStreaming, limitReached, error, sendDoubt }
}
```

4.2 -- Frontend: src/screens/AskAI.tsx

```
import { useState, useRef, useEffect } from 'react'
import { useAuth } from '../context/AuthContext'
import { useDoubtStream } from '../hooks/useDoubtStream'
import { getDailyUsage, submitFeedback, generateVisual } from '../services/api'
import { AnimatePresence, motion } from 'framer-motion'

export default function AskAIScreen() {
  const { token, user } = useAuth()
  const [subject, setSubject] = useState('Physics')
  const [className] = useState(10)  // from user profile in real app
  const [input, setInput] = useState('')
  const [usage, setUsage] = useState({ count: 0, limit: 10 })
  const [visualHtml, setVisualHtml] = useState<string | null>(null)
  const [loadingVisual, setLoadingVisual] = useState(false)
  const bottomRef = useRef<HTMLDivElement>(null)

  const { messages, isStreaming, limitReached, error, sendDoubt } =
    useDoubtStream(token, subject, className)

  // Load usage count on mount
  useEffect(() => {
    if (token) getDailyUsage(token).then(setUsage)
  }, [token])

  // Auto-scroll to bottom on new messages
  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' })
  }, [messages])

  async function handleSend() {
    if (!input.trim() || isStreaming) return
    const q = input.trim()
    setInput('')
    await sendDoubt(q)
    // Refresh usage count
    if (token) getDailyUsage(token).then(setUsage)
  }

  async function handleVisual(concept: string) {
    if (!token) return
    setLoadingVisual(true)
    const html = await generateVisual(token, concept, subject, className)
    setVisualHtml(html)
    setLoadingVisual(false)
  }
```

```
  async function handleFeedback(sessionId: string, helpful: boolean, reason?:
string) {
    if (token) await submitFeedback(token, sessionId, helpful, reason)
  }

  const showCounter = usage.count >= 5

  return (
    <div className='flex flex-col h-screen max-w-3xl mx-auto'>

      {/* Subject selector */}
      <div className='flex gap-2 p-4 border-b overflow-x-auto'>
        {['Physics','Chemistry','Maths','Biology','English'].map(s => (
          <button key={s} onClick={() => setSubject(s)}
            className={`px-3 py-1.5 rounded-lg text-xs font-medium whitespace-nowrap
              ${subject === s ? 'bg-teal-50 text-teal-700 border border-teal-200'
                : 'text-gray-500 hover:bg-gray-50'}`}>
            {s}
          </button>
        ))}
      </div>

      {/* Messages */}
      <div className='flex-1 overflow-y-auto p-4 space-y-4'>
        {messages.length === 0 && (
          <div className='text-center py-16'>
            <div className='text-4xl mb-3'>&#129302;</div>
            <p className='text-gray-500 text-sm'>Ask me anything from your CBSE
syllabus</p>
          </div>
        )}

        <AnimatePresence>
        {messages.map(msg => (
          <motion.div key={msg.id}
            initial={{ opacity: 0, y: 8 }} animate={{ opacity: 1, y: 0 }}
            className={`flex ${msg.role === 'user' ? 'justify-end' : 'justify-
start'}`}>

            <div className={`max-w-[85%] rounded-2xl px-4 py-3 text-sm leading-
relaxed
              ${msg.role === 'user'
                ? 'bg-teal-600 text-white rounded-br-sm'
                : 'bg-white border rounded-bl-sm shadow-sm'}`}>

              {/* Thinking state */}
              {msg.role === 'assistant' && msg.streaming && msg.content === '' && (
                <div className='flex gap-1 py-1'>
                  {[0,1,2].map(i => (
                    <div key={i} className='w-1.5 h-1.5 bg-gray-400 rounded-full
                      animate-bounce' style={{ animationDelay: `${i*0.15}s` }} />
                  ))}
                </div>
              )}

              {/* Response content */}
              <p className='whitespace-pre-wrap'>{msg.content}</p>

              {/* Streaming cursor */}
              {msg.streaming && msg.content && (
                <span className='inline-block w-0.5 h-4 bg-teal-500 ml-0.5 animate-
pulse' />
              )}

              {/* Action chips -- shown after streaming ends */}
              {msg.role === 'assistant' && !msg.streaming && msg.content && (
                <div className='mt-3 flex flex-wrap gap-2 border-t pt-3'>
                  {['Explain simpler','Quiz me','Show exam answer'].map(chip => (
                    <button key={chip}
```

```
                      onClick={() => sendDoubt(chip + ': ' +
messages[messages.indexOf(msg)-1]?.content)}
                      className='text-xs px-2.5 py-1 bg-gray-50 text-gray-600
                        border rounded-lg hover:bg-teal-50 hover:text-teal-700
                        hover:border-teal-200 transition-colors'>
                      {chip}
                    </button>
                  ))}
                  <button
                    onClick={() => handleVisual(messages[messages.indexOf(msg)-
1]?.content || '')}
                    className='text-xs px-2.5 py-1 bg-teal-50 text-teal-700
                      border border-teal-200 rounded-lg hover:bg-teal-100
transition-colors'>
                    Explain visually
                  </button>

                  {/* Quality signals */}
                  <div className='ml-auto flex gap-2 items-center'>
                    <button onClick={() => handleFeedback(msg.id, true)}
                      className='text-gray-400 hover:text-teal-500 transition-colors
text-xs'>
                      Helpful
                    </button>
                    <button onClick={() => handleFeedback(msg.id, false,
'inaccurate')}
                      className='text-gray-400 hover:text-red-400 transition-colors
text-xs'>
                      Not helpful
                    </button>
                  </div>
                </div>
              )}
            </div>
          </motion.div>
        ))}
        </AnimatePresence>

        {/* Visual explain overlay */}
        {(visualHtml || loadingVisual) && (
          <div className='bg-white border rounded-2xl p-4 shadow-sm'>
            <div className='flex items-center justify-between mb-3'>
              <span className='text-xs font-medium text-teal-600'>Visual
explanation</span>
              <button onClick={() => setVisualHtml(null)}
                className='text-gray-400 hover:text-gray-600 text-xs'>
                Close
              </button>
            </div>
            {loadingVisual ? (
              <p className='text-sm text-gray-400 text-center py-6'>Generating
visual...</p>
            ) : (
              <iframe
                srcDoc={visualHtml!}
                className='w-full rounded-lg border-0'
                style={{ minHeight: 200, maxHeight: 400 }}
                sandbox='allow-scripts'
              />
            )}
          </div>
        )}

        {error && <p className='text-red-500 text-sm text-center'>{error}</p>}
        <div ref={bottomRef} />
      </div>

      {/* Input bar */}
      {limitReached ? (
        <div className='p-4 border-t bg-amber-50'>
```

```
          <p className='text-sm text-amber-700 text-center'>
            You have used your 10 free doubts today.
          </p>
          <button className='mt-2 w-full text-sm text-teal-600 hover:underline'>
            Upgrade to unlimited (Rs.99/month)
          </button>
        </div>
      ) : (
        <div className='p-4 border-t bg-white'>
          <div className='flex gap-2 items-end'>
            <div className='flex-1 relative'>
              <textarea
                value={input}
                onChange={e => setInput(e.target.value)}
                onKeyDown={e => { if (e.key === 'Enter' && !e.shiftKey) {
e.preventDefault(); handleSend() } }}
                placeholder='Ask anything from your CBSE syllabus...'
                rows={1}
                className='w-full border rounded-xl px-4 py-3 pr-16 text-sm
                  resize-none focus:outline-none focus:ring-2 focus:ring-teal-300'
              />
              {showCounter && (
                <span className={`absolute right-3 bottom-3 text-xs
                  ${usage.count >= 9 ? 'text-red-400' : usage.count >= 8 ? 'text-
amber-500' : 'text-gray-400'}`}>
                  {usage.count}/10
                </span>
              )}
            </div>
            <button onClick={handleSend} disabled={isStreaming || !input.trim()}
              className='bg-teal-600 text-white px-4 py-3 rounded-xl text-sm font-
medium
                hover:bg-teal-700 disabled:opacity-40 transition-colors'>
              {isStreaming ? '...' : 'Ask'}
            </button>
          </div>
        </div>
      )}
    </div>
  )
}
```

Part 5 -- Home Screen (Week 3)

The home screen is the daily re-engagement hub. It shows the student's real XP, streak, subject mastery, and an AI recommendation card. Wire each module from the database one at a time.

5.1 -- Backend: GET /api/user/home-data

One endpoint returns all the data the home screen needs. Fewer round trips, simpler loading state.

```
// Add to server/routes/user.ts
user.get('/home-data', async (c) => {
  const u = await getUserFromToken(c.req.header('Authorization') ?? null)
  if (!u) return c.json({ error: 'Unauthorized' }, 401)

  const [profile, xpRows, streaks, mastery, recentDoubts] = await Promise.all([
    supabase.from('profiles').select('*').eq('id', u.id).single(),
    supabase.from('student_xp').select('amount').eq('student_id', u.id),
    supabase.from('student_streaks').select('*').eq('student_id', u.id).single(),
    supabase.from('subject_mastery').select('*').eq('student_id', u.id),
    supabase.from('doubt_sessions')
      .select('subject, created_at')
      .eq('student_id', u.id)
      .order('created_at', { ascending: false })
      .limit(3),
  ])

  const totalXP = (xpRows.data || []).reduce((s: number, r: any) => s + r.amount, 0)
  const todayXP = (xpRows.data || []).filter((r: any) => {
    const today = new Date().toISOString().split('T')[0]
    return r.created_at?.startsWith(today)
  }).reduce((s: number, r: any) => s + r.amount, 0)

  return c.json({
    profile: profile.data,
    totalXP,
    todayXP,
    dailyGoal: 50,
    streak: streaks.data || { current_streak: 0, longest_streak: 0 },
    mastery: mastery.data || [],
    recentSubjects: (recentDoubts.data || []).map((d: any) => d.subject),
  })
})
```

5.2 -- Frontend: src/screens/Home.tsx

```
import { useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'

interface HomeData {
  profile: { name: string; class_level: number }
  totalXP: number
  todayXP: number
  dailyGoal: number
  streak: { current_streak: number; longest_streak: number }
  mastery: { subject: string; accuracy_percent: number }[]
  recentSubjects: string[]
}
```

```
function XPRing({ value, max }: { value: number; max: number }) {
  const r = 28, circ = 2 * Math.PI * r
  const fill = Math.min(value / max, 1) * circ
  return (
    <svg width='72' height='72' viewBox='0 0 72 72'>
      <circle cx='36' cy='36' r={r} fill='none' stroke='#E5E7EB' strokeWidth='4' />
      <circle cx='36' cy='36' r={r} fill='none' stroke='#0D9488' strokeWidth='4'
        strokeDasharray={`${fill} ${circ}`}
        strokeLinecap='round' transform='rotate(-90 36 36)' />
      <text x='36' y='36' textAnchor='middle' dominantBaseline='central'
        className='text-xs font-semibold fill-gray-700'>{value}</text>
    </svg>
  )
}

export default function HomeScreen() {
  const { token } = useAuth()
  const nav = useNavigate()
  const [data, setData] = useState<HomeData | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    if (!token) return
    fetch('/api/user/home-data', {
      headers: { Authorization: `Bearer ${token}` }
    }).then(r => r.json()).then(setData).finally(() => setLoading(false))
  }, [token])

  if (loading) return (
    <div className='flex items-center justify-center h-screen'>
      <div className='animate-pulse text-teal-500 text-sm'>Loading your
session...</div>
    </div>
  )

  if (!data) return null

  const hour = new Date().getHours()
  const greeting = hour < 12 ? 'Good morning' : hour < 17 ? 'Good afternoon' : 'Good
evening'

  return (
    <div className='max-w-4xl mx-auto p-6 space-y-4'>

      {/* Header */}
      <div className='flex items-center justify-between'>
        <div>
          <h1 className='text-xl font-semibold'>{greeting},
{data.profile.name.split(' ')[0]}!</h1>
          <p className='text-sm text-gray-500'>Class {data.profile.class_level}</p>
        </div>
        <div className='flex items-center gap-3'>
          <XPRing value={data.todayXP} max={data.dailyGoal} />
          <div className='flex items-center gap-1 bg-orange-50 text-orange-600
            px-3 py-1.5 rounded-xl text-sm font-medium'>
            <span>&#128293;</span>
            <span>{data.streak.current_streak} days</span>
          </div>
        </div>
      </div>

      {/* AI Recommendation card */}
      <div className='bg-navy-900 bg-gray-900 text-white rounded-2xl p-5'>
        <p className='text-xs text-teal-400 font-medium mb-2'>AI RECOMMENDATION FOR
TODAY</p>
        {data.mastery.length > 0 ? (
          <div>
            <p className='text-sm leading-relaxed mb-4'>
              {(() => {
```

```
                const weak = data.mastery.find(m => m.accuracy_percent < 70)
                return weak
                  ? `Your accuracy in ${weak.subject} is ${weak.accuracy_percent}%.
Let me help you fix that.`
                  : `Great progress! Keep your ${data.streak.current_streak}-day
streak going.`
              })()}
            </p>
            <button onClick={() => nav('/ask')}
              className='bg-teal-500 text-white px-4 py-2 rounded-xl text-sm font-
medium
                hover:bg-teal-400 transition-colors'>
              Start practice
            </button>
          </div>
        ) : (
          <div>
            <p className='text-sm text-gray-300 mb-4'>
              Ask your first doubt to get personalised recommendations.
            </p>
            <button onClick={() => nav('/ask')}
              className='bg-teal-500 text-white px-4 py-2 rounded-xl text-sm font-
medium'>
              Ask first doubt
            </button>
          </div>
        )}
      </div>

      {/* Subject mastery rings */}
      {data.mastery.length > 0 && (
        <div>
          <p className='text-xs font-semibold text-gray-400 uppercase tracking-wider
mb-3'>
            Subject mastery
          </p>
          <div className='grid grid-cols-3 gap-3 sm:grid-cols-5'>
            {data.mastery.map(m => (
              <button key={m.subject}
                onClick={() => nav(`/ask?subject=${m.subject}`)}
                className='bg-white border rounded-xl p-3 text-center hover:border-
teal-200
                  hover:bg-teal-50 transition-colors'>
                <XPRing value={m.accuracy_percent} max={100} />
                <p className='text-xs text-gray-600 mt-1'>{m.subject}</p>
              </button>
            ))}
          </div>
        </div>
      )}

      {/* Quick actions */}
      <div className='grid grid-cols-2 gap-3'>
        <button onClick={() => nav('/ask')}
          className='bg-white border rounded-2xl p-4 text-left hover:border-teal-200
            hover:bg-teal-50 transition-colors'>
          <p className='text-sm font-medium'>Ask AI</p>
          <p className='text-xs text-gray-500 mt-1'>Get any doubt solved</p>
        </button>
        <button onClick={() => nav('/practice')}
          className='bg-white border rounded-2xl p-4 text-left hover:border-blue-200
            hover:bg-blue-50 transition-colors'>
          <p className='text-sm font-medium'>Practice</p>
          <p className='text-xs text-gray-500 mt-1'>5 targeted questions</p>
        </button>
      </div>
    </div>
  )
}
```

Part 6 -- Worksheet Generator Screen (Week 4)

The worksheet screen has two modes: Custom (teacher configures everything) and Paper Mimic (teacher uploads a CBSE exam PDF). Both call the backend and show validated questions with NCERT badges.

6.1 -- Frontend: src/screens/Worksheet.tsx

```
import { useState } from 'react'
import { useAuth } from '../context/AuthContext'
import { generateWorksheet, uploadMimicPDF } from '../services/api'

type Mode = 'custom' | 'mimic'

interface Question {
  number: number
  question: string
  answer: string
  marks: number
  validated: boolean
  regenerated?: boolean
}

interface WorksheetResult {
  title: string
  sections: { name: string; questions: Question[] }[]
}

export default function WorksheetScreen() {
  const { token } = useAuth()
  const [mode, setMode] = useState<Mode>('custom')
  const [loading, setLoading] = useState(false)
  const [loadingStep, setLoadingStep] = useState('')
  const [result, setResult] = useState<WorksheetResult | null>(null)

  // Custom mode state
  const [subject, setSubject] = useState('Physics')
  const [chapter, setChapter] = useState('')
  const [className] = useState(10)
  const [totalQ, setTotalQ] = useState(10)

  // Mimic mode state
  const [pdfFile, setPdfFile] = useState<File | null>(null)

  const steps = ['Creating questions...', 'Checking against NCERT...',
'Finalising...']

  async function handleCustomGenerate() {
    if (!token) return
    setLoading(true)
    setResult(null)
    for (const step of steps) {
      setLoadingStep(step)
      await new Promise(r => setTimeout(r, 800))
    }
    try {
      const ws = await generateWorksheet(token, {
        subject, className, chapter, totalQuestions: totalQ,
        questionTypes: ['MCQ','Short answer'], difficulty: 'mixed'
      })
      setResult(ws)
    } finally {
```

```
      setLoading(false)
      setLoadingStep('')
    }
  }

  async function handleMimicGenerate() {
    if (!token || !pdfFile) return
    setLoading(true)
    setResult(null)
    setLoadingStep('Reading exam format...')
    await new Promise(r => setTimeout(r, 1000))
    setLoadingStep('Generating similar questions...')
    try {
      const ws = await uploadMimicPDF(token, pdfFile, subject, className, 20)
      setResult(ws)
    } finally {
      setLoading(false)
      setLoadingStep('')
    }
  }

  return (
    <div className='max-w-5xl mx-auto p-6'>
      <h1 className='text-xl font-semibold mb-6'>Worksheet generator</h1>

      {/* Mode tabs */}
      <div className='flex gap-2 mb-6'>
        {(['custom','mimic'] as Mode[]).map(m => (
          <button key={m} onClick={() => setMode(m)}
            className={`px-4 py-2 rounded-xl text-sm font-medium border transition-
colors
              ${mode === m
                ? 'bg-teal-50 border-teal-300 text-teal-700'
                : 'border-gray-200 text-gray-500 hover:bg-gray-50'}`}>
            {m === 'custom' ? 'Custom' : 'Paper Mimic'}
          </button>
        ))}
      </div>

      <div className='grid grid-cols-1 md:grid-cols-2 gap-6'>

        {/* Config panel */}
        <div className='bg-white border rounded-2xl p-5'>
          <div className='space-y-4'>
            <div>
              <label className='text-xs font-medium text-gray-500 uppercase
tracking-wide'>Subject</label>
              <select value={subject} onChange={e => setSubject(e.target.value)}
                className='mt-1 w-full border rounded-lg px-3 py-2 text-sm'>
                {['Physics','Chemistry','Mathematics','Biology','English'].map(s =>
(
                  <option key={s}>{s}</option>
                ))}
              </select>
            </div>

            {mode === 'custom' ? (
              <>
                <div>
                  <label className='text-xs font-medium text-gray-500
uppercase'>Chapter</label>
                  <input value={chapter} onChange={e => setChapter(e.target.value)}
                    placeholder='e.g. Electricity'
                    className='mt-1 w-full border rounded-lg px-3 py-2 text-sm' />
                </div>
                <div>
                  <label className='text-xs font-medium text-gray-500
uppercase'>Questions: {totalQ}</label>
                  <input type='range' min={5} max={30} value={totalQ}
```

```
                    onChange={e => setTotalQ(Number(e.target.value))}
                    className='mt-1 w-full' />
                </div>
                <button onClick={handleCustomGenerate} disabled={loading ||
!chapter}
                  className='w-full bg-teal-600 text-white py-2.5 rounded-xl text-sm
                    font-medium disabled:opacity-40 hover:bg-teal-700'>
                  Generate worksheet
                </button>
              </>
            ) : (
              <>
                <div>
                  <label className='text-xs font-medium text-gray-500 uppercase'>
                    Upload CBSE board paper (PDF)
                  </label>
                  <input type='file' accept='.pdf'
                    onChange={e => setPdfFile(e.target.files?.[0] || null)}
                    className='mt-1 w-full text-sm text-gray-500 file:mr-3 file:py-
1.5
                      file:px-3 file:rounded-lg file:border-0 file:text-xs
file:font-medium
                      file:bg-teal-50 file:text-teal-700' />
                  {pdfFile && <p className='text-xs text-gray-400 mt-
1'>{pdfFile.name}</p>}
                </div>
                <button onClick={handleMimicGenerate} disabled={loading || !pdfFile}
                  className='w-full bg-teal-600 text-white py-2.5 rounded-xl text-sm
                    font-medium disabled:opacity-40 hover:bg-teal-700'>
                  Generate similar questions
                </button>
              </>
            )}
          </div>
        </div>

        {/* Preview panel */}
        <div className='bg-white border rounded-2xl p-5 min-h-48'>
          {loading && (
            <div className='flex flex-col items-center justify-center h-48 gap-3'>
              <div className='w-6 h-6 border-2 border-teal-500 border-t-transparent
                rounded-full animate-spin' />
              <p className='text-sm text-gray-500'>{loadingStep}</p>
            </div>
          )}

          {result && (
            <div>
              <h3 className='font-semibold text-gray-900 mb-4'>{result.title}</h3>
              {result.sections?.map((section, si) => (
                <div key={si} className='mb-6'>
                  <p className='text-xs font-semibold text-gray-400 uppercase mb-3'>
                    {section.name}
                  </p>
                  <div className='space-y-3'>
                    {section.questions?.map((q, qi) => (
                      <div key={qi} className='border rounded-xl p-3'>
                        <div className='flex items-start justify-between gap-2'>
                          <p className='text-sm text-gray-800 flex-1'>
                            <span className='font-medium text-gray-500 mr-
2'>Q{q.number || qi+1}.</span>
                            {q.question}
                          </p>
                          <div className='flex items-center gap-2 shrink-0'>
                            <span className='text-xs text-gray-
400'>{q.marks}M</span>
                            {q.validated && (
                              <span className='text-xs bg-green-50 text-green-700
                                border border-green-200 px-2 py-0.5 rounded-full'>
                                NCERT verified
```

```
                              </span>
                            )}
                          </div>
                        </div>
                      </div>
                    ))}
                  </div>
                </div>
              ))}
              <button onClick={() => window.print()}
                className='mt-4 w-full border border-teal-300 text-teal-600 py-2
                  rounded-xl text-sm hover:bg-teal-50 transition-colors'>
                Download PDF
              </button>
            </div>
          )}

          {!loading && !result && (
            <div className='flex items-center justify-center h-48'>
              <p className='text-sm text-gray-400'>
                Configure and generate a worksheet
              </p>
            </div>
          )}
        </div>
      </div>
    </div>
  )
}
```

Part 7 -- Loading, Error, and Skeleton States

Every data-dependent screen must show a skeleton while loading. Never show a blank screen. These reusable components work across all screens.

7.1 -- src/components/Skeleton.tsx

```
export function SkeletonLine({ w = 'full', h = '4' }: { w?: string; h?: string }) {
  return <div className={`h-${h} w-${w} bg-gray-200 rounded animate-pulse`} />
}

export function SkeletonCard() {
  return (
    <div className='bg-white border rounded-2xl p-4 space-y-3'>
      <SkeletonLine w='2/3' />
      <SkeletonLine />
      <SkeletonLine w='4/5' />
    </div>
  )
}

export function HomeScreenSkeleton() {
  return (
    <div className='max-w-4xl mx-auto p-6 space-y-4'>
      <div className='flex items-center justify-between'>
        <div className='space-y-2'><SkeletonLine w='32' /><SkeletonLine w='20' h='3'
/></div>
        <div className='w-16 h-16 bg-gray-200 rounded-full animate-pulse' />
      </div>
      <SkeletonCard />
      <div className='grid grid-cols-3 gap-3'>
        {[1,2,3].map(i => <SkeletonCard key={i} />)}
      </div>
    </div>
  )
}
```

7.2 -- src/components/ErrorBoundary.tsx

```
import { Component, type ReactNode } from 'react'

interface Props { children: ReactNode }
interface State { hasError: boolean; error?: Error }

export class ErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false }

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error }
  }

  render() {
    if (this.state.hasError) return (
      <div className='flex flex-col items-center justify-center min-h-48 p-8 text-
center'>
        <p className='text-gray-500 text-sm mb-3'>Something went wrong.</p>
        <button onClick={() => this.setState({ hasError: false })}
          className='text-teal-600 text-sm hover:underline'>Try again</button>
      </div>
```

```
    )
    return this.props.children
  }
}
```

Part 8 -- Complete 8-Week Build Checklist

Frontend and backend tasks side by side. Build the backend task before wiring the frontend.

Week	Backend task	Frontend task
W1	Server starts. /api/health returns 200. All 3 SQL migrations run. First NCERT PDF uploaded.	Project created. Tailwind configured. AuthContext + ProtectedRoute working. Login screen renders.
W2	Full /api/ai/doubt route (dual-loop + RAG + cache). curl test returns streaming response.	useDoubtStream hook. AskAI screen wired. Real streaming works. Usage cap counter visible.
W3	Photo doubt route (Llama Vision). /api/ai/visual route. /api/ai/feedback + /api/ai/flag routes.	Quality signals (thumbs up/down, report). Visual explain iframe. Action chips after response.
W4	Onboarding route (POST /api/user/onboarding). Home data route (GET /api/user/home-data).	Onboarding 3-step flow. Home screen all modules with real data. XP ring, streak flame.
W5	Practice route (POST /api/ai/practice). Evaluate route (POST /api/ai/evaluate).	Practice MCQ session. Answer evaluation feedback card. XP toast on correct answer.
W6	Worksheet route with validation agent. Paper Mimic route. Admin NCERT upload route.	Worksheet generator screen (both modes). NCERT verified badges. Admin upload panel at /admin.
W7	Check Groq handles 80% of calls. Cache hit rate in response_cache table. Sentry + PostHog.	Error boundary on all screens. Skeleton loading states. Empty states for all data-dependent views.
W8	Deploy backend to Railway. Verify all routes work on prod URL.	Deploy frontend to Vercel. Point padee.ai domain. Test end-to-end on prod. First 3 school pilots.